

● ① 本システム開発の技術的背景

- a) SMPTE ST2110による放送局の基幹システムをこれから構築
- b) 各システムはST2110で映像音声アンシラリデータをIPネイティブで相互疎通する
 - i) この他に制御のストリームはout of bandで相互接続する
 - ii) ここで言う各システムの定義は、回線センター、報道サブ、制作サブとなり、管理する組織やSLAが異なるものである
 - iii) 「ゾーン」という考え方
 - (1) 複数ある報道サブ、制作サブをそれぞれ、「報道ゾーン」「スポーツ/エンタメゾーン」と定義する
 - (2) このため、前述の管理する組織やSLAについて分類することがゾーンの分類と一致する
 - (3) この後の本書については、全てゾーン単位で考えることにする
- c) NMOSによる映像音声アンシラリの疎通制御をすることを前提とする
- d) 本システムは実在する本物のSenderを別の名前で名乗って、SDPの内容はReal と全く同じものを常に同期した状態で、他ゾーンに伝達することを目指す。このため、「Alias Sender」と命名する。

● ② 本システム導入の目的と改善効果

- a) ゾーン間の制御ネットワークのセキュリティ確保
- b) ゾーンを跨ぐNMOSの制御を制御ネットワーク経由で動的に行う。つまりSDPファイルの登録を静的に行わなわない
- c) これにより、回線センターでNodeのメンテナンスなどを行いSDPの内容が変わっても、他ゾーンの制御システムは自動的に追従することを可能にする
- d) NMOSに完全準拠する場合、各Senderは1つのRDSに対してのみRegistry APIによる登録を行う。これに対して、本システムでは各ゾーンに存在するRDSに「Alias Sender」として代替的に登録することにより、回線センターの物理機器内に論理的に存在するReal Senderを、「Real Senderであるかのようにふるまう複数のAlias Sender」として複数ゾーンのRDSに登録することを目指す。
- e) 回線センターで高密度ゲートウェイ(以下GW)を採用した場合、NodeやDeviceが「大きな塊」になってしまい、実際に番組制作の運用部門である、各ゾーンでは非常に扱いにくい。Sender1つ単位でNMOSでのLabel管理を行い、個別システムゾーンで運用者が人間的にわかりやすいLabel名でオペレートすることで運用ミスを減らす

● ③ 本システムのゴール

- a) AMWA NMOS IS-04とIS-05の詳細仕様は以下URLから参照して適用すること
<https://specs.amwa.tv/is-04/>
<https://specs.amwa.tv/is-05/>
- b) 別項のハードウェアに本システムを構築し、前章に記載の環境で、実際の放送機器と疎通を行う
- c) 本書に機材の考え方が、本番の基幹システムにふさわしいものなのか、本開発に進む前に判断をする
- d) このためシステムの安定性やセキュリティは考慮せず、NMOSのSenderとしてのふるまいがきちんとできることにのみ注力する
- e) 最終的に、別章で定めた「他システムのブロードキャストコントローラー」が本システムを介して、Receiverが「Real Senderに対してNode APIやConnection APIを送信することなく」本物のSenderと全く同一のSDP情報を「Alias Sender」から取得し、他システムのReceiverがこのSDP情報を元にIGMP Joinを出すことがゴールとなる。

● ④ 用語集と定義

- a) NMOS
 - i) ST2110で映像音声アンシラリを異メーカー間で相互に接続制御する。本システムではNMOSのお作法に完全に準拠する
- b) IS-04

- i) NMOSにおいて、Registration API、Query API、Node APIを元に、機器の登録と発見を行う。本システムではIS-04のお作法に完全に準拠する。
- c) IS-05
 - i) NMOSにおいて、Connecion APIなどを用いてSenderのSDP情報を取得してReceiverがSenderにST2110でIGMP Join (Version3のみを対象) するためのもの。本システムではIS-05のお作法に完全に準拠する。
- d) RDS
 - i) IS-04に則り、Node情報を保存し他機器にその情報を伝達する。
 - ii) 本システムはRDS機能は持たないが、RDSとNMOSのお作法に則って情報を取得また伝達する
- e) SDP
 - i) Senderが自身の映像音声アンシラリのフォーマットおよびIGMP Joinに必要なIPレベルの情報をまとめたもの。本システムでも使用するが、本物のSenderの情報は一切加工しないことを大前提とする。
- f) Node
 - i) カメラや音声装置など、目に見える機器物体そのもの。NMOSにおける最大の単位となる。
- g) Device
 - i) Node内のSenderやReceiverのグループを指す。NMOSにおいて一般的には一意のルールがないと理解してよい。
- h) Sender
 - i) ST2110において、マルチキャストストリームを送出する仮想のコネクタである。一般的にはマルチキャストアドレスを送信先アドレスに設定する。
 - ii) 1つのDevice内に複数のSenderを持つNodeが一般的には多い。
- i) Receiver
 - i) ST2110において、マルチキャストストリームを受信する仮想のコネクタである。
 - ii) 本システムにおいては、Receiverは存在しないが、他システムのReceiverに必要な不可欠な情報を提供することが本システムの目的である。
- j) Connection API
 - i) IS-05において、SenderからSDPを取得しReceiverにその情報を送信してSDPの内容を元に受信設定を変更しIGMP JoinをさせるためのAPIである。
- k) Registration API
 - i) IS-04において、Nodeが自身の情報をRDSに登録するためのAPIである。
 - ii) ポート番号は自由であるが、本システムではユーザーがGUIなどで設定する。
- l) Query API
 - i) IS-04において、本システムのようなシステムやNMOSコントローラー、ブロードキャストコントローラーが、RDSの情報を取得するためのAPIである。
 - ii) ポート番号は、接続先RDSに合わせる必要があり、本システムではユーザーがGUIなどで設定する。
 - iii) 本書における「WebSocket」とは、IS-04 Query APIの標準機能である/subscriptions!によるリアルタイム通知の仕組みを指す
- m) Node API
 - i) IS-04における「P2Pモード」で動作する他ゾーンのNMOSコントローラに対応する。
 - ii) ただし、他ゾーンRDSはIPアドレスの別セグメントに存在するため、mDNSやDNS-SDでのシステムからの自動登録は不要で、相手NMOSコントローラから本システムへの静的な状態取得に対応するだけで問題ない。
 - iii) ポート番号は、本システムでユーザーがGUIなどで設定し、相手NMOSコントローラーが合わせることをとする。
- n) ブロードキャストコントローラー
 - i) NMOSに限らない、様々なプロトコルを駆使して、ST2110の放送設備の制御を行うシステムである。
- o) ゾーン
 - i) 本システムを利用する放送局特有の考え方である。
 - ii) マスタープレイアウト、信号分配、制作系スタジオ、報道系スタジオ、などSLAや管理部署、運用者などが異なる分野と、ST2110の放送機器の物理的な管理分野を同期させるための

考え方である。つまり「マスターゾーン」「回線センターゾーン」「制作系スタジオゾーン」「報道系スタジオゾーン」などを物理的および論理的に分離する。

- iii) このゾーンは不要な放送局もあれば、4つ5つと増える放送局もある。
- iv) このように管理部門や管理品質が異なるゾーンを設けるが、ゾーンをまたいで全社で共有すべきゾーン内の特定のNodeおよびSender(Receiverは含まない)を、NMOSに則って自由に使用できて、かつ、各ゾーン内のブロードキャストコントローラーで運用上のトラブルを未然に防ぐ、ことはNMOSの標準的なモデルでは考慮されていない。

p) Alias Unit Server

- i) 本システムが入る物理的なハードウェアを指す

q) Alias Node

- i) 各ゾーン内のローカルRDSにRegistryする仮想的なNode。
- ii) NMOSの標準的なお作法とは異なり、本システムのAlias Nodeは静的登録しか考慮しないため、1つのAlias Nodeが複数のRDSに登録することも可能とする。この場合、相手RDSからは通常のNMOSのNodeと同じように見えるようにふるまう。
- iii) 本システムにおいては、Nodeの役割を持つが、実際に映像音声アンシラリは生成や受信を行わない。あくまでNodeのようにふるまい、他機器に必要な「制御データ」を伝達することが目的である。
- iv) 本システムにおいては、1つの物理サーバーの中に複数の仮想的なNodeを格納することができる。これはユーザーで任意に設定が可能である。この仮想的なNodeは論理的に分離されている。

r) Alias Device

- i) 後述するAlias Connectorを複数まとめて管理するためのグルーピング。NMOSにおけるDeviceのお作法に準拠する。
- ii) 本システムにおいては、1つのNodeの中に複数のDeviceをユーザーが任意に好きな数で割り当てることができて、またDeviceの名前(ラベル)のユーザーが管理運用しやすいように設定することができる。
- iii) 同じAlias Deviceを複数のAlias Nodeに割り当てることが可能である。

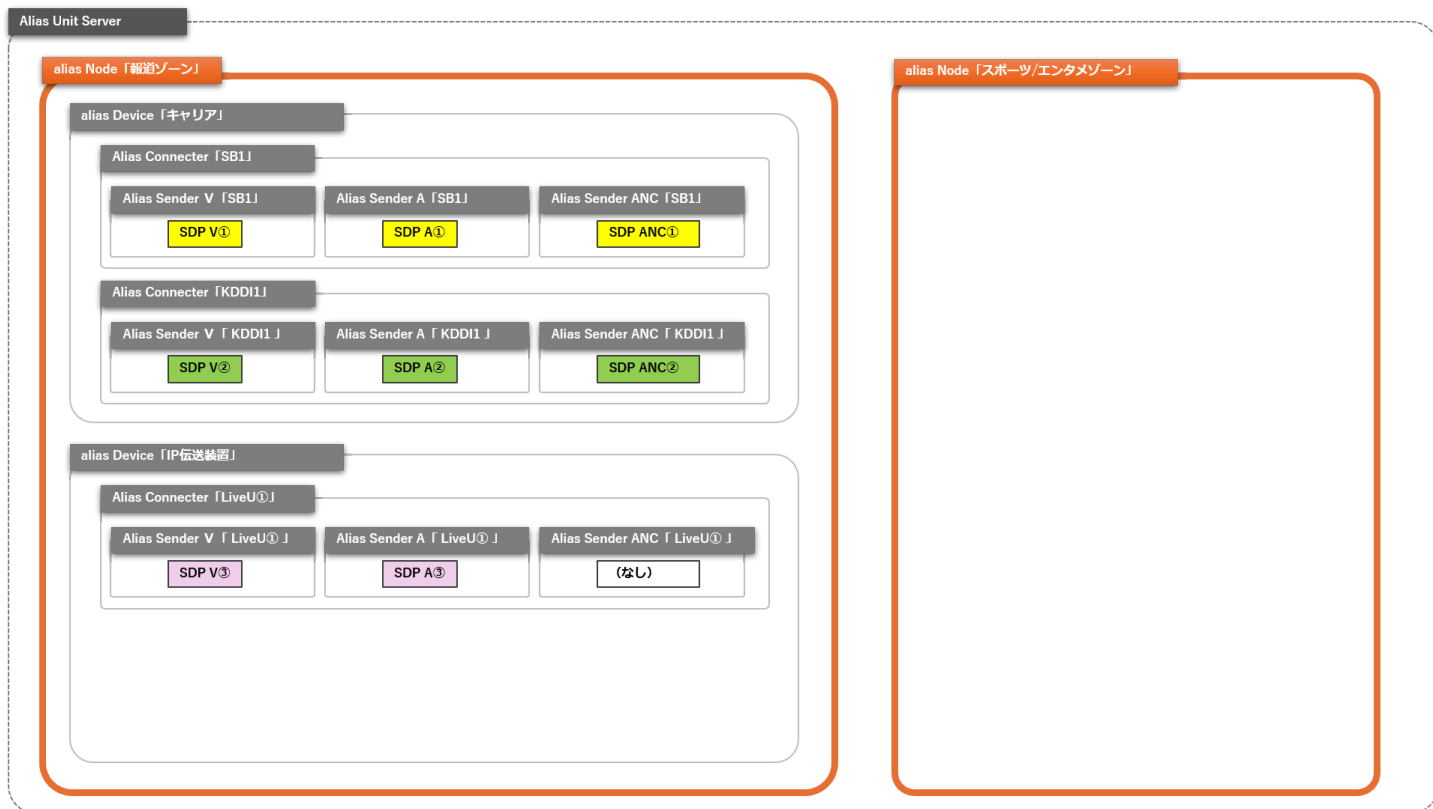
s) Alias Connector

- i) SDIライクにV/A/ANCの最大3つのかたまり。Connector単位で「SB1」など名前(label)を設定することができる。本システム内のローカルな考え方であり、Connectorという単位はNMOSには存在しないため、他システムへのNMOSでの広告には使用しない。
- ii) 他ゾーンRDSに対しては、前述のDeviceの単位での広告となり、ここではこのAlias Connectorという単位は広告しない。

t) Alias Sender

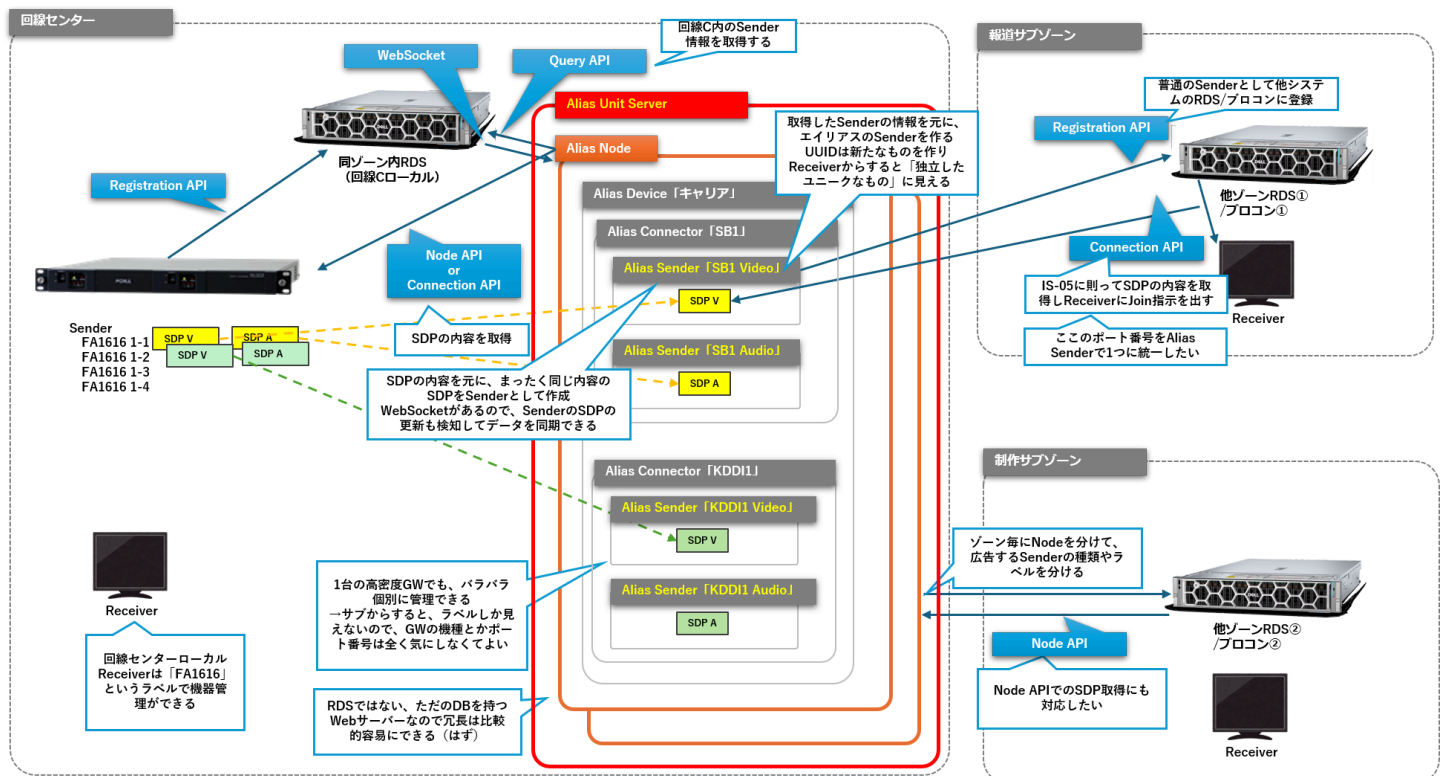
- i) 本物のSenderから作成する。上記の「ゾーン」で記述した、ゾーンをまたいだSenderの共有をNMOSを用いて行うための別名のSenderである。
- ii) 別名であるため、別にオリジナルのSenderが存在し、これはReal Senderである。Alias SenderはこのReal Senderと全く同じSDPの情報をリアルタイム同期したうえでデータ管理する。
- iii) 本システム内の仮想的なNodeの中に作成したDeviceの中に存在するものである。
- iv) ラベルをユーザーが管理しやすい任意の文字列で設定可能であり、これは登録先のRDSに登録されるものである。
- v) NMOSのお作法に則り、UUIDを設定する。
- vi) NMOSのお作法に則り、[http://\(IPアドレス\):\(ポート番号\)/x-nmos/](http://(IPアドレス):(ポート番号)/x-nmos/) で始まるURLで外部機器からアクセスできる。この配下のディレクトリ構造はNMOSのIS-04およびIS-05の両方に完全に準拠する。
- vii) あるReal Senderを2つのゾーンのRDSに登録したい場合、本システムの仮想的なNodeを2つ用意することが基本である。つまり、本システムの物理サーバーの中には、同じAlias Senderが2つ以上、複製して複数存在することが可能であるが、NodeやDeviceが異なるため、相手側機器に影響を及ぼすことはない。これはこのNodeが複数ある場合でも、NodeがIS-04のRegistration APIで登録するRDSは別物であるからである。

● 本装置内の各機能の概要図



- ⑤ハードウェアなど動作環境 ※検証のための環境であり製品としての完全環境ではない
 - a) Ubuntu Server 24.04.4が動くハードウェアを使用
 - b) 本システム以外は存在しない
 - c) WebGUIでユーザーがクライアント端末から各種設定変更を行う
 - d) バックエンドはPythonを使用
 - e) NMOSの標準プロトコルに完全に準拠する。基本的にはIS-04はVer1.3、IS-05はVer1.1を標準とするが、下位Verの周辺システムにも対応するため、IS-04は1.1～1.3、IS-05は1.0～1.1を本システムでは用意する
 - f) この基本思想を実現するためには、OSSを使用することは問題ない
 - g) 後述する各種DBについては、SQL Liteを使用する
 - h) Dockerコンテナ化する。ポート番号は本システムで管理するがIPアドレスはOSでの管理とする。
(Dockerはホストネットワークモード(--network host)で起動し、コンテナがホストのIPアドレスとポートを直接使用する。これにより動的なポート追加時にもコンテナ側のポートマッピング設定変更は不要とする)
 - i) 本Serverは「Alias Unit Server」と命名する

● ⑥本システムと周辺システムの関係



- 上記概要図において、本システムは「Alias Unit Server」と赤で書かれた箱＝物理的なハードウェア、を指している。前章の通り、1台のハードウェアに本システムのみが存在している
- 周辺のRDSやNodeについては、NMOS-CPPなど標準的なOSSIによるNMOS機器となり、本システム外となる
- 回線センター/報道サブゾーン/制作サブゾーンなど、各ゾーンにはそれぞれNMOSに準拠したRDS/ブロードキャストコントローラー/Node(Device/Sender/Receiverを含む)が存在する。
- 回線センターにあるNodeは回線センターの回線RDSに登録される。Alias Unit Serverは回線RDSからQueryおよびWebSocketで形でReal Senderの情報とリアルタイムに同期する。
- Alias Unit ServerはReal Senderに関するSDPファイルを、NMOSに準拠する形でReal Senderに問い合わせ取得する
- Alias Unit Serverでは各ゾーンのRDSに対して登録するための各ゾーンAlias Nodeを作成する。Alias Nodeの名称(label)はユーザーが任意で変更できるようにする

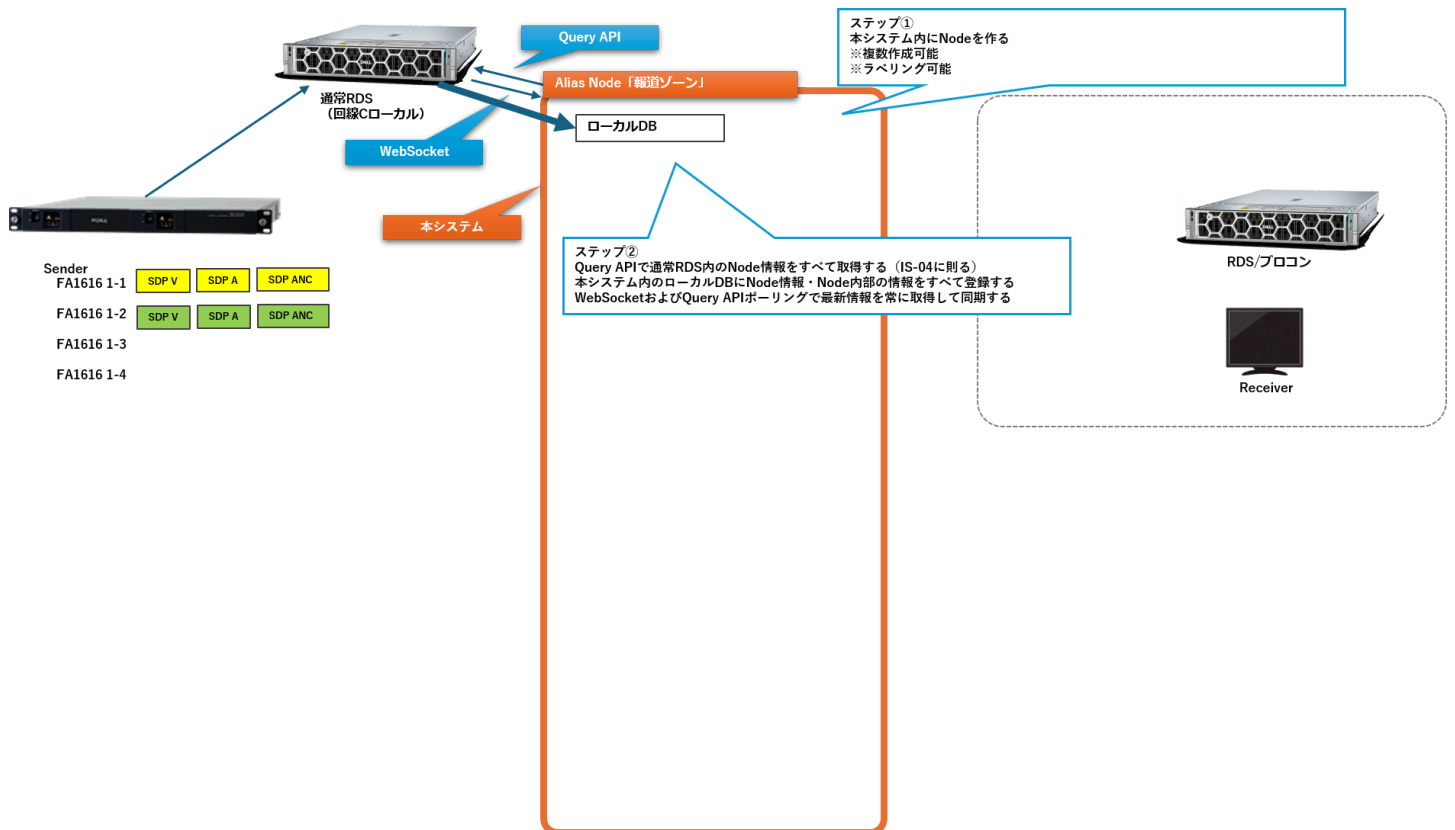
● ⑦システムの分類概要

- ① NMOS IS-04 RDSからのデータ受信部
 - NMOS IS-04に則り、RDS①からNode情報を受信してローカルDBに情報を登録する
 - また常にRDS①とローカルDBの状態を同期させる
- ② 「Alias Node」部
 - Alias Nodeのデータを作成し、後述のX-Yクロスポイントで複数のAlias Deviceと紐づけ
 - Alias Node自体も複数保持可能
 - Alias Device内には複数のAlias Connectorを作成する
(HD-SDIライクの枠組みとするため、1Alias Connectorの中にはAudio・Video・Ancillaryの3Alias Senderが存在できるようにする)
 - Alias Connectorの内部には、3つのAlias Senderが存在する
(1) Video, Audio, ANCになるが、ユーザーにより任意の素材をNullに設定できる
(2) これら3つのAlias SenderをReal Senderと紐づける
 - これらのデータを管理する「Alias Node DB」機能が必要
(1) ローカルDBとは別テーブルを想定

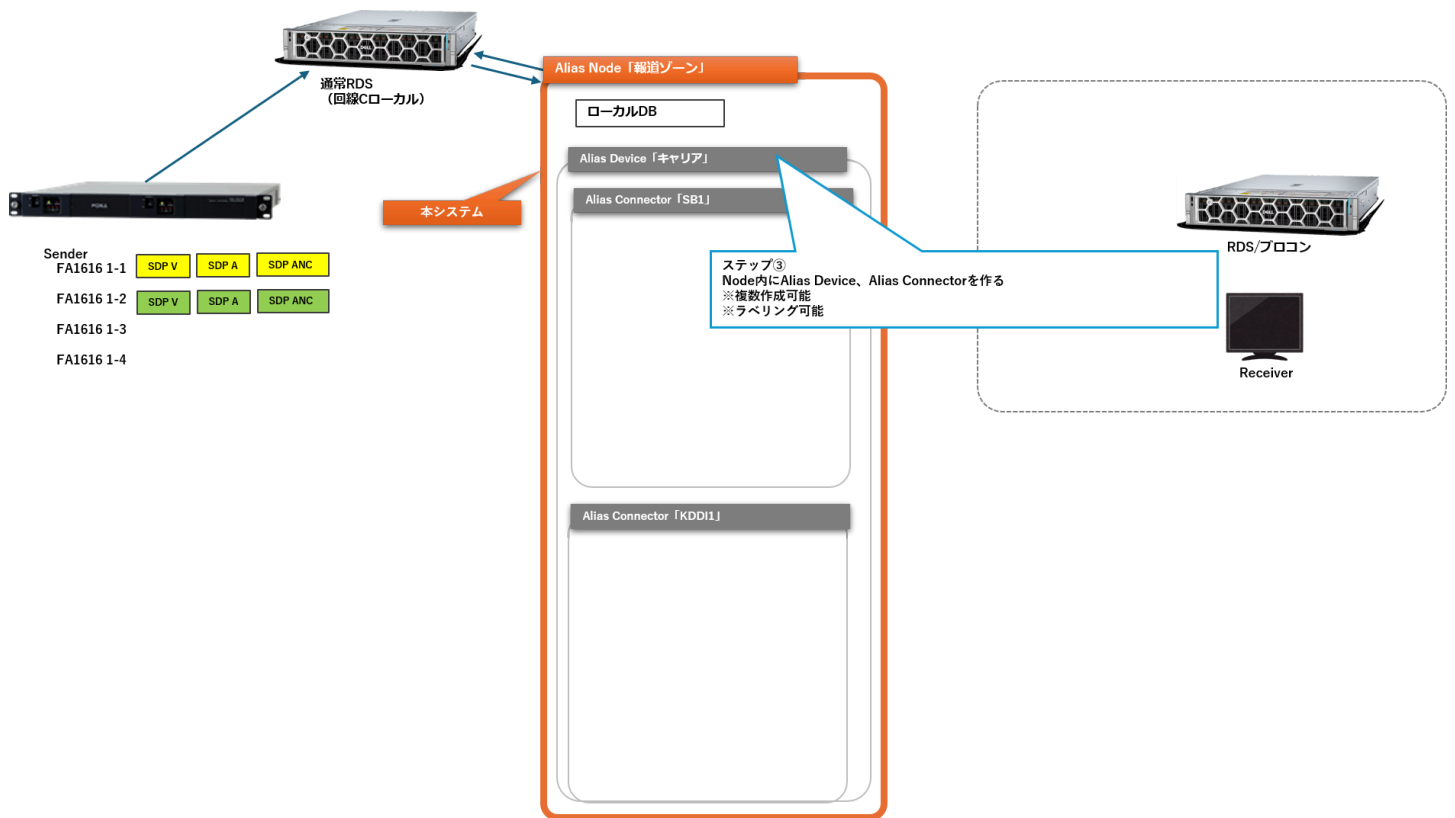
- vi) Node APIに対応するため、他システムからREST APIでアクセスするためのAPIを用意する。SDPの取得などに対応し、必要なリクエストに対応できるようにする。
- c) ③ Alias Sender機能
 - i) Real SenderとAlias Senderを1対1で紐づける機能
 - (1) NodeやDeviceではなく、Sender単位である
 - ii) この2つのSender内のSDPは常に同期する(リアルタイム同期)
 - (1) MasterはReal Senderであり、Alias Senderは必ずSlaveという関係
 - iii) Alias Senderが保持するデータ構造(SDP本体、UUID、紐付け元Senderの参照、ラベル、所属Device/ゾーン情報など)
 - iv) Real Senderが何らかの原因で同一ゾーンRDSから消えた場合、本システムのAlias Senderは「オフライン」となり、他システムのRDS②への登録もOFFにする。復活したときには速やかにRDS②への登録も復活する
- d) ④ NMOS IS-04に則り、本システムのNodeを、他システムのRDS②に登録
 - i) IS-04に則って登録。
 - ii) 他システムのNMOSコントローラーなどからのIS-05に則ったAPIでSDPの情報を送る
- e) ⑤ システム管理WebGUI
 - i) 各種設定
 - ii) ポート番号などのシステムに関わる部分
 - iii) OSレベルの「情報表示」

● ⑧本システムの運用想定および本システムの内部フロー解説

- a) ステップ0(初期設定) 各ゾーンのRDS情報の登録(Registration APIを送信する連携先の設定)
- b) ステップ① 各ゾーンのRDSに登録するAlias Nodeの作成
Alias Nodeの接続先のRDSはAlias Unit Serverでテーブル管理する
- c) ステップ② 同一ゾーンRDSからのReal Node情報の取得



- d) ステップ③ Alias Device、Alias Connectorの作成
Alias Device、Alias Connectorのテーブルを作成、変更、削除可能とする。これはReal NodeやReal Deviceには依存しないAlias Unit Server独自のテーブルで管理する。Alias Deviceのlabelとdescriptionはユーザーが変更可能とする。使い方は「label:回線名」「description:回線種別」とする

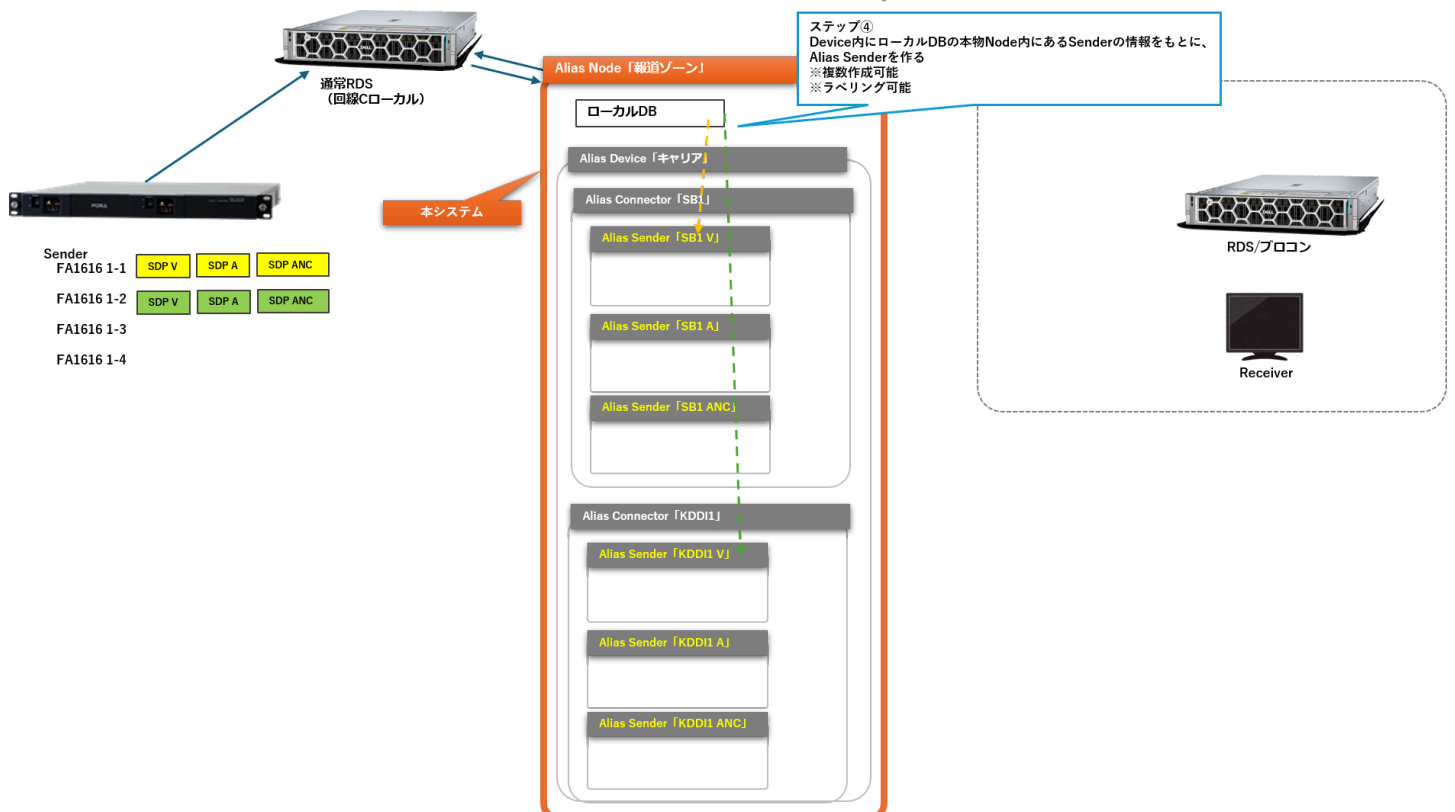


e) ステップ④ Alias Senderの作成

Real Senderと1対1で紐づける形でAlias Senderを作成する

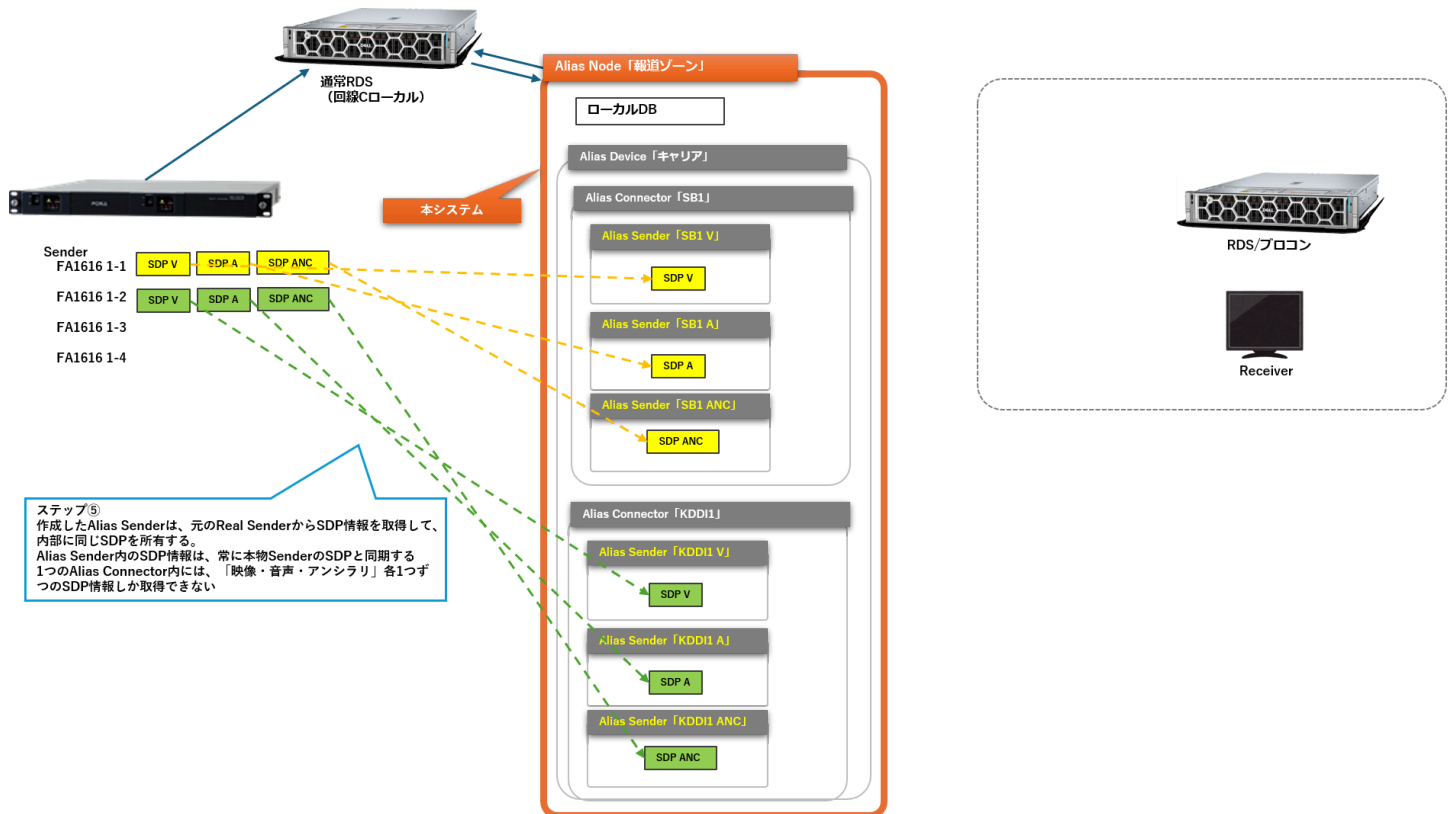
Alias ConnectorとAlias Senderの紐づけ作業

- これは本ステップに包括されるが、Alias Senderの紐づけ先Alias Connectorは変更が可能である
- 紐づけを行った結果、Alias Senderのlabelは「Alias Connector label_V/A/ANC」とすること。
※V/A/ANCはSenderのVideo/Audio/Ancillaryの種類により決定。



f) ステップ⑤ Real SenderからSDP情報を取得し、Alias SenderのSDP情報と同期する

- i) 本プロセスは、本システム稼働内は常時動き、常にReal Senderをマスターとしてリアルタイムで同期し続ける

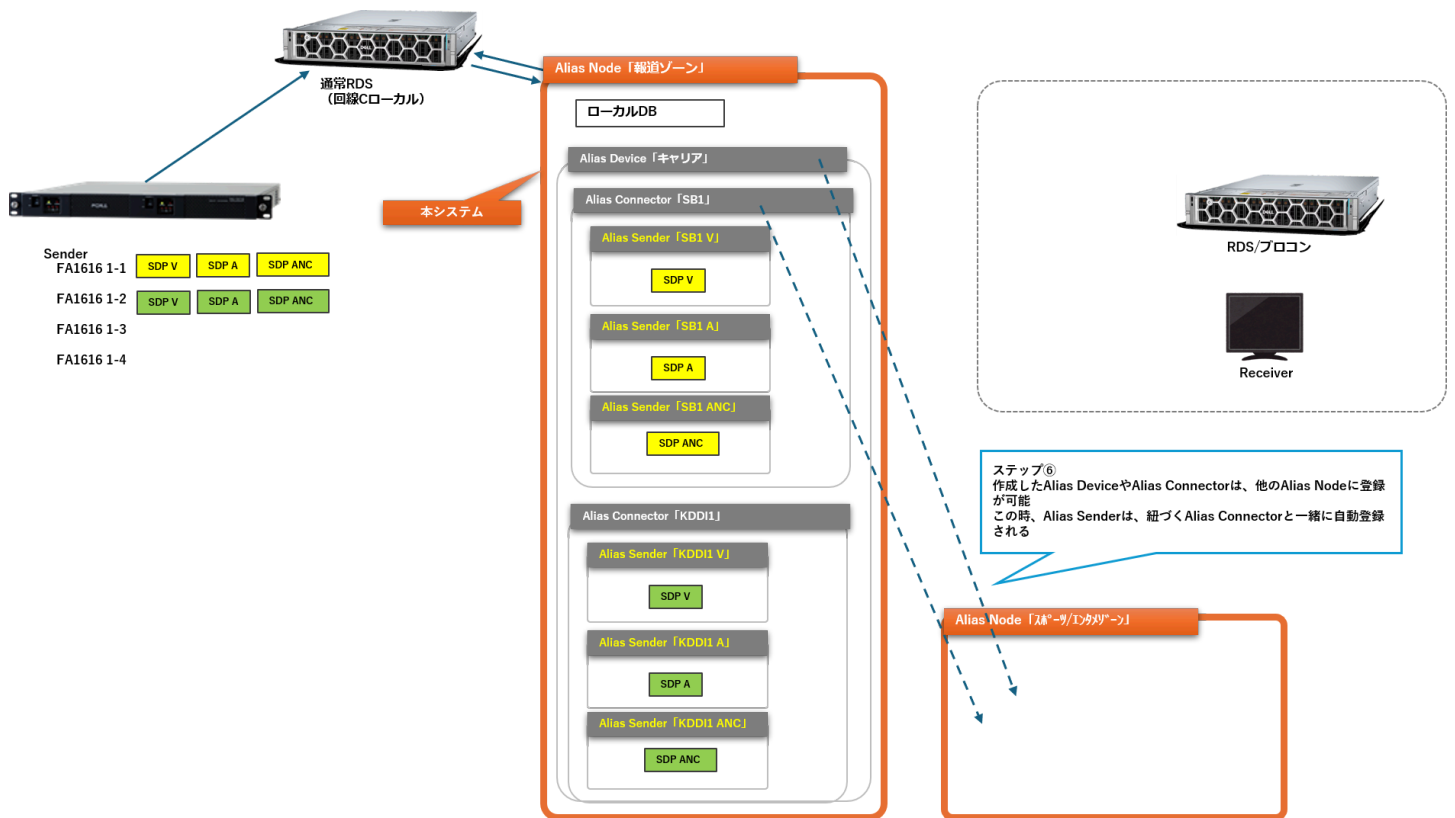


- g) ステップ⑥ Alias NodeとAlias DeviceとAlias Connectorの紐づけ作業

この作業を行うためのGUIは

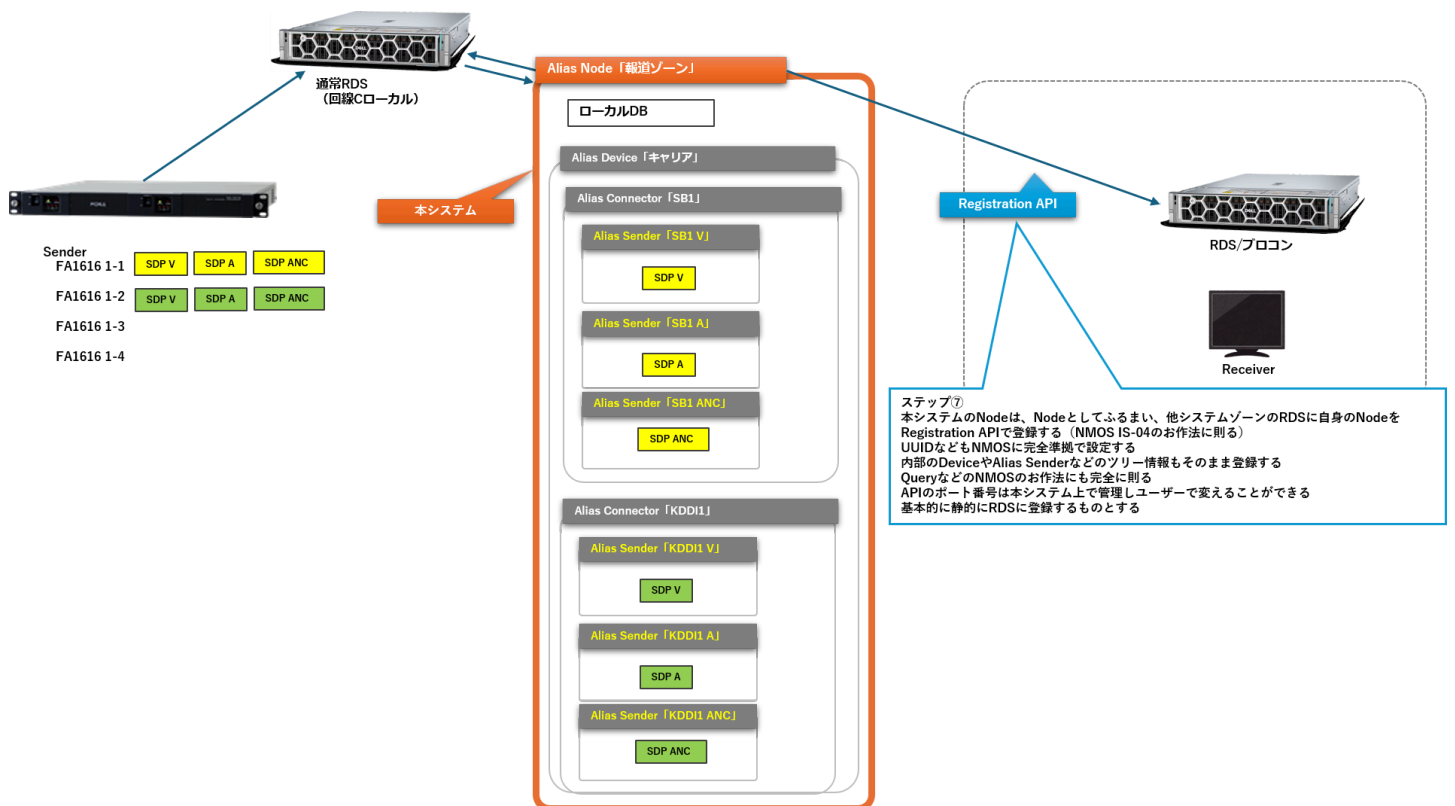
X: Alias Node Y: Alias Device & Alias Connectorとした 基盤目のようなX-Yクロスポイントの選択/非選択で

行う。1 Alias Nodeに対して多数のAlias Deviceが選択可能。また1つのAlias Device内のAlias Connectorは複数のAlias Nodeに選択可能。この作業を持って、Alias Unit ServerではAlias Node/Alias Device/Alias Sender/Alias Sender(に対するSDPファイル)の紐づけが完了し、Alias Nodeが完成する

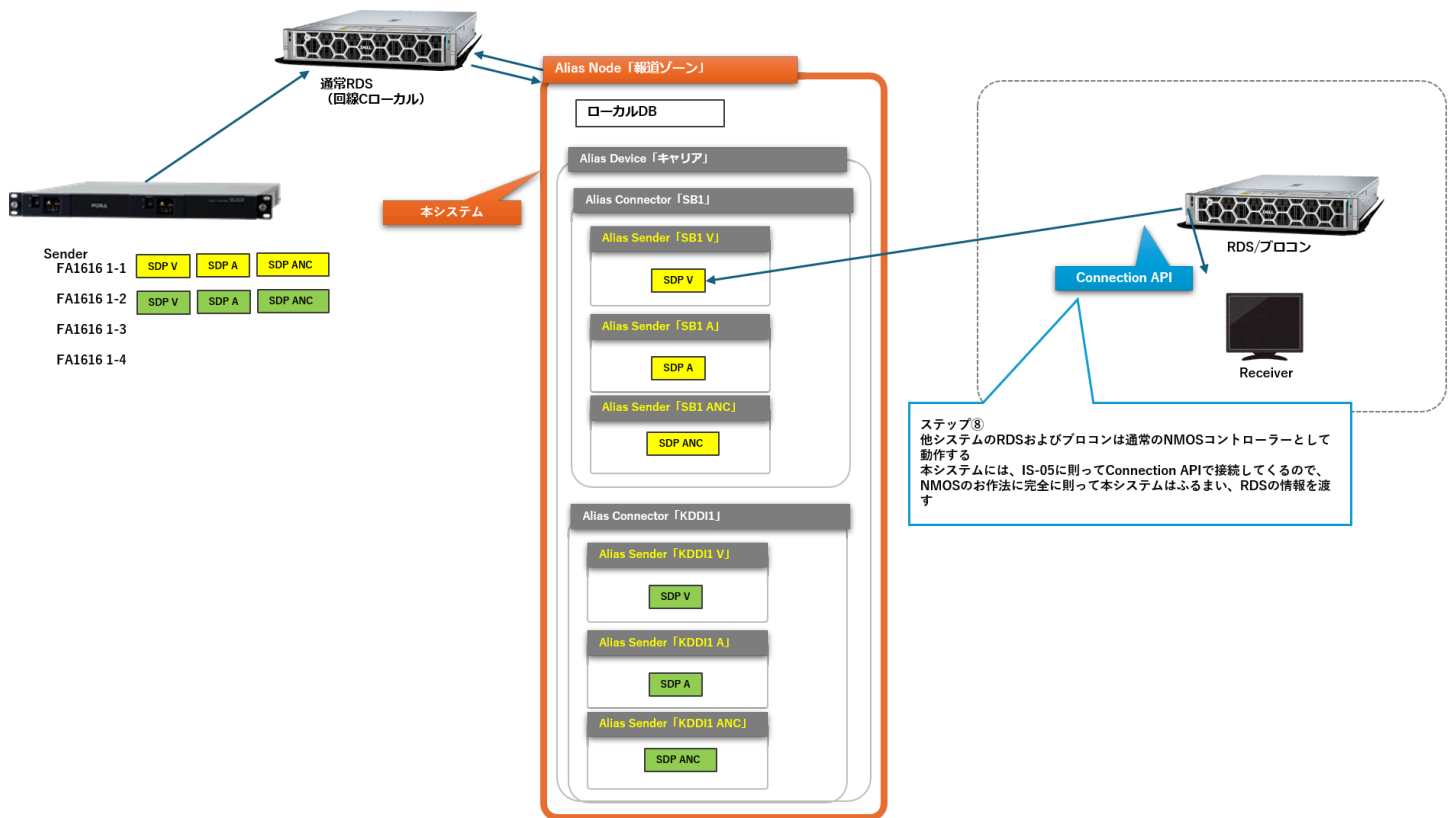


h) ステップ⑦ 他ゾーンRDSに登録

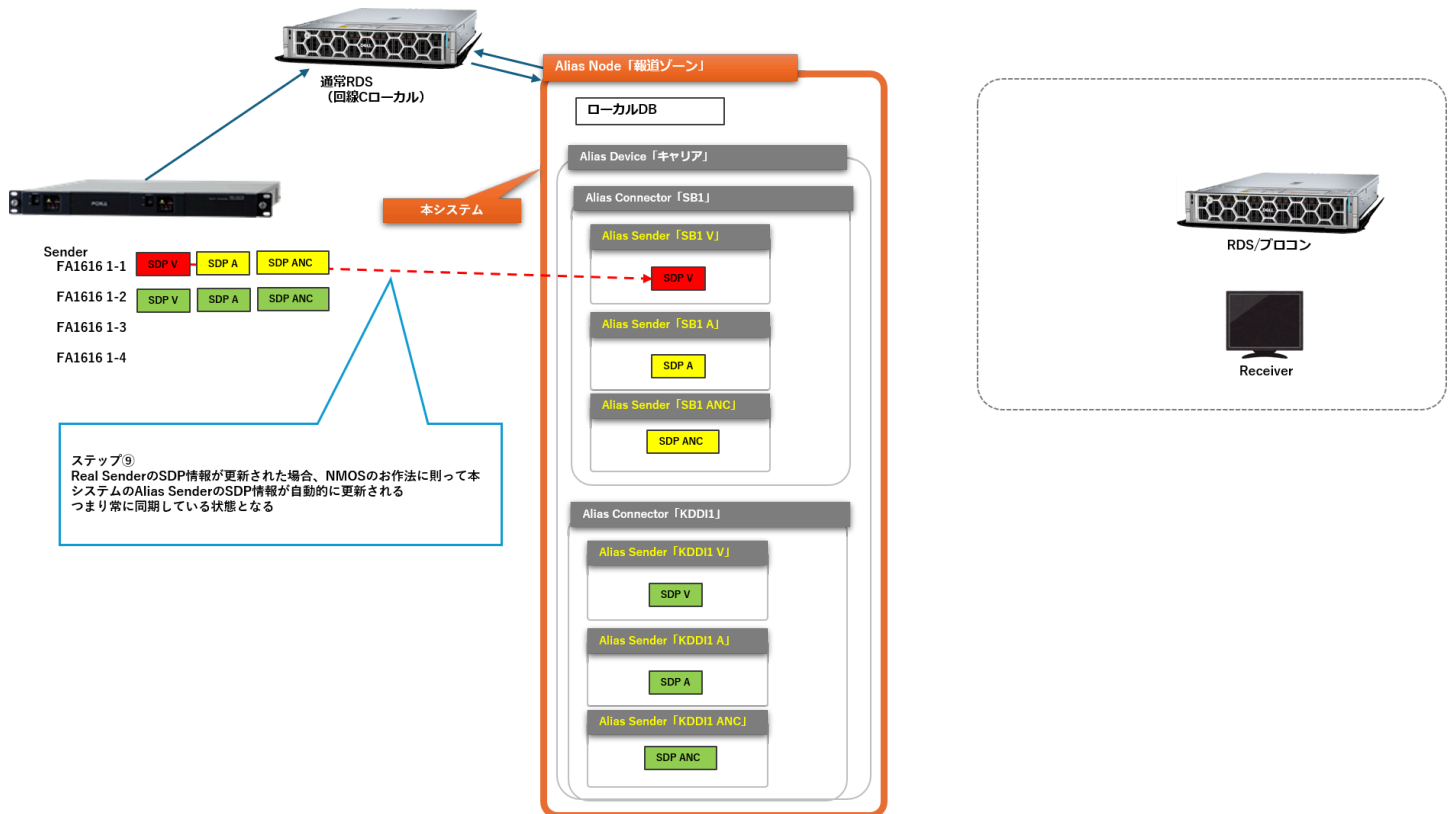
- i) これは、IS-04に則る。つまり他ゾーンの相手RDSから見ると、ハートビートなどを含めて、Nodeのようにふるまい続ける。あくまでNMOSのREST APIでの話であり、ST2110のストリームの送信自体は、Real Senderが行う。また、Real Senderは常に自身の情報を、同一RDSにNMOSのお作法に則って登録し続ける。



i) ステップ⑧ IS-05での制御



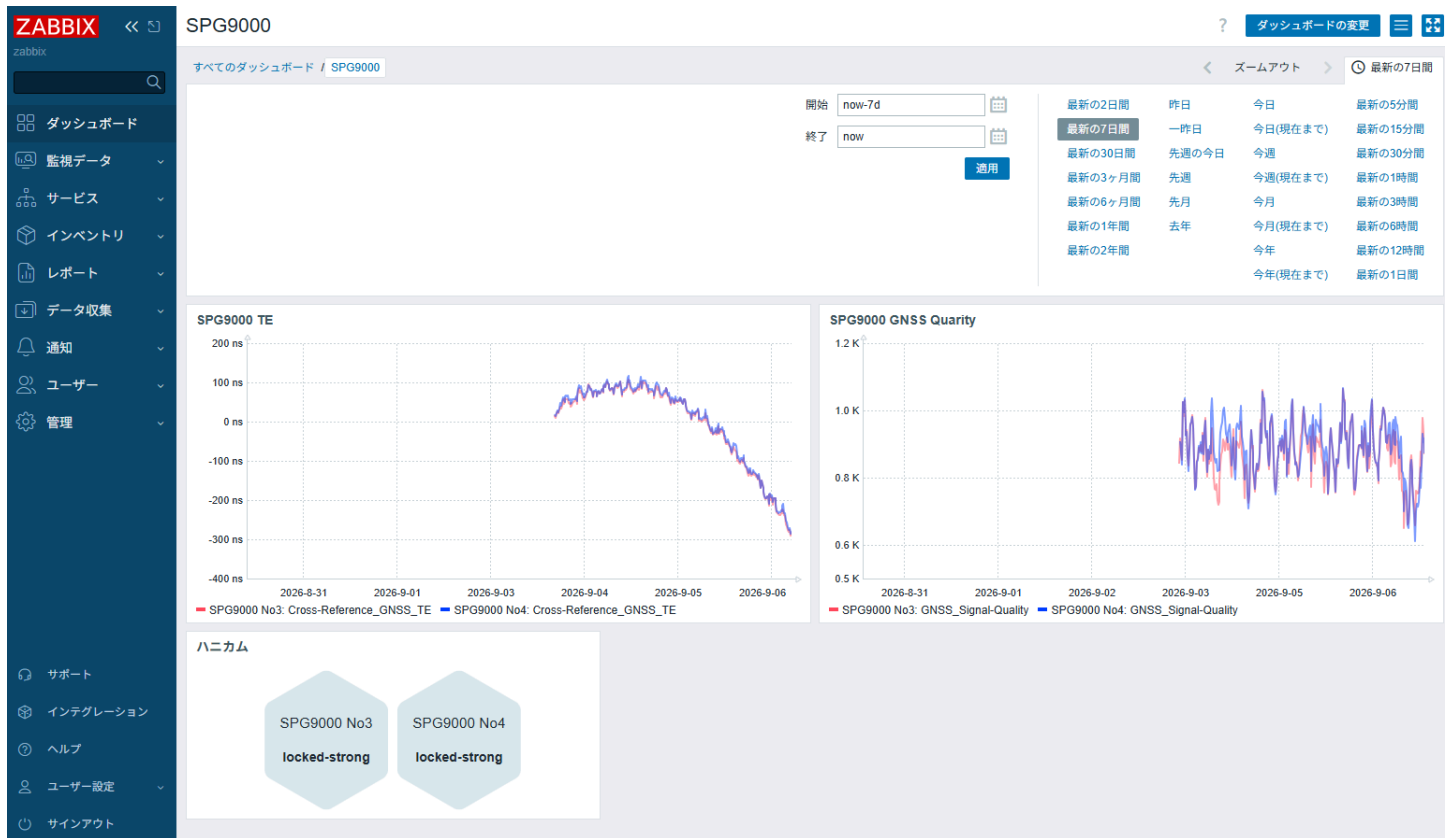
j) ステップ⑨ Real SenderのRDSとの同期



● ⑨WebGUI詳細

- デザインはgrafanaのようなダークグレーを基調としたものにする
- 左ペインにはZabbixのようにメニューを設ける
 - ダッシュボード
 - 状態の確認を行うためだけの画面
 - RDS登録管理
 - 設定管理
 - Alias Nodeの作成管理

- (1) 設定管理
- iv) Alias Device/Connectorの作成管理
 - (1) 設定管理
- v) Alias Senderの作成管理
 - (1) 設定管理
- vi) システム設定



c) 各メニューの詳細

i) 「ダッシュボード」メニュー

(1) メインペインは縦で複数の”行”に分割する。なるべく文字を小さくしてたくさんの情報を一気にみられるようにしたい

(a) ペイン① RDS接続情報

(i) 同一ゾーンRDS

1. 接続状態

2. 接続先RDSのIPアドレスとポート番号とIS-04のバージョン

(ii) 他ゾーンRDS ①、②、③・・・と必要に応じて縦に増やす。ペイン内でスクロールして全ペインが1つのGUIに収まるようにする

1. 接続状態

2. 接続先RDSのIPアドレスとポート番号とIS-04のバージョン

(b) ペイン② 本システムDB状態表示

(i) 同一ゾーンRDSから取得したNodeの一覧表示

1. Nodeをクリックすると、同一タブ内の別ウィンドウ(画面いっぱいに出す)で、Node内のDeviceとSenderとReceiver状態が表示される

(c) ペイン③ OS情報、本システム設定情報

(i) IPアドレスの表示

(d) ペイン④ Alias Node情報① ※Nodeが増えたらペインを増やす

(i) Alias Device一覧

1. 特定のDeviceをクリックすると中のConnectorが「広がる」。再度クリックして表示を収納することもできる
2. Alias Connectorをクリックすると、中の3つのAlias Senderが同一タブ内の別小Windowで表示される。

- a. Real Senderとの同期状態、ラベル、S,G状態 を1行で出す
 - b. ここでSDPファイルの内容の表示やダウンロードが可能
- ii) 「RDS登録管理」メニュー
 - (1) 同一ゾーン RDS 情報
 - Query APIの送信先IPアドレス、ポート、API Ver.を設定する
 - (i) ON/OFF
 - (ii) IPアドレス、ポート番号
 - (iii) Query APIのバージョン(プルダウン)
 - (2) 他ゾーン RDS 情報
 - Registry APIの送信先IPアドレス、ポート、API Ver.を設定する
 - つまりRegistry APIはStaticのみ
 - 他ゾーンRDSは複数登録可能
 - (a) Registry API設定
 - (i) ON/OFF
 - (ii) IPアドレス、ポート番号
 - (iii) Registration APIのバージョン(プルダウン)
 - (b) Node API設定 ※Node単位で行やペインを分ける
 - (i) ON/OFF
 - (ii) ポート番号
- iii) 「Alias Nodeの作成管理」メニュー
 - (1) Alias Node
 - 他ZoneRDSにAlias Nodeを紐づける形。
 - Alias NodeのNode labelを設定する。
 - Alias Nodeの削除は、紐づいたRDSがないことが条件。
 - 1つのRDSに対して複数のAlias Nodeは選択不可
 - 1つのAlias Nodeに対して複数のRDSは選択可能
- iv) 「Alias Device/Connectorの作成管理」メニュー
 - (1) Alias Device
 - (a) 作成と削除
 - (i) Alias Deviceのlabelとdescription設定
「label:回線名」「description:回線種別」とする
 - (b) Alias Node とAlias Deviceの紐づけ設定(X-Yクロスポイント画面)
 - (2) Alias Connector
 - (a) 作成と削除
 - (b) Connector名設定
- v) 「Alias Sender設定」メニュー
 - (1) Query APIにて取得したNode/Device/Senderから、作成対象のNodeとDeviceをプルダウンで選択する
 - (a) 本システムDB内の、同一ゾーン内RDSのNodeを選択
 - (i) Deviceを選択
 - 1. 対象のReal Senderを選択する
 - a. ラベルは、Alias Connectorを元に自動作成される
※Alias Senderとの紐づけ後はこの画面でのSender labelの変更不可
 - b. 他ゾーンRDSに広告するためのdescriptionも設定する
 - c. ※Alias Senderとの紐づけ後もこの画面でのSender descriptionは変更可能
 - d. 映像音声アンシラリの分類を手動で設定する

※初期値はQuery APIの情報から反映すること

2. この時、Real Senderを見つけやすくするために、文字列フィルターや映像音声アンシリリでフィルターできるようにする

vi) 「システム設定」メニュー

(1) データベースを汎用フォーマットファイルでのエクスポートとインポート

(2) OSのIPアドレスを取得して「表示」する ※設定変更などは行わう必要はない

● ⑩データモデル定義

1. RealSender(Real Sender情報 / ローカルDBキャッシュ)

同一ゾーンRDSからIS-04経由で取得した、本物のNode/Device/Senderの情報をキャッシュするテーブル。1.

項目名	型	説明
id	UUID (PK)	ローカル管理用の内部ID
nmos_node_id	UUID	Real Node側のNMOS Node ID
nmos_device_id	UUID	Real Device側のNMOS Device ID
nmos_sender_id	UUID	Real SenderのNMOS Sender ID
nmos_sender_label	string	Real Senderのラベル(例: FA1616 1-1)
sdp_raw	text	取得した生のSDP内容
media_type_detected	enum(video/audio/ancillary)	SDPから自動判定したメディア種別
status	enum(online/offline)	同一ゾーンRDS上での存在確認状態
last_seen_at	datetime	RDSから最後に確認できた日時

2. AliasNode(本システム内の仮想Node)

項目名	型	説明
id	UUID (PK)	NMOS Node IDとして外部公開されるID
alias_node_label	string	ユーザー設定のNode名
alias_node_description	string	任意の説明文
node_api_enabled	boolean	Node API/Connection API 提供のON/OFF(新規)
node_api_port	int	このNodeが待ち受けるポート番号。Node APIとConnection APIを同一ポート・ <i>/x-nmos/</i> 配下の別パスで共通提供する(新規)
created_at	datetime	作成日時

3. AliasDevice(Connectorをまとめるグループ)

項目名	型	説明
id	UUID (PK)	NMOS Device IDとして外部公開
alias_device_label	string	ユーザー設定のDevice名(例:「SB1」「渋谷情カメ」)
alias_device_description	string	ユーザー設定のDevice種別名(例:「キャリア」「IP」)
created_at	datetime	作成日時

4. NodeDeviceAssignment(Node×DeviceのX-Yクロスポイント紐づけ)

項目名	型	説明
id	UUID (PK)	
node_id	UUID (FK → AliasNode)	紐づけ先のAlias Node
device_id	UUID (FK → AliasDevice)	紐づけ対象のAlias Device
assigned_at	datetime	紐づけ設定日時

1つのAlias Nodeに複数のAlias Deviceを紐づけ可能、1つのAlias Deviceを複数のAlias Nodeに紐づけ可能(⑧ステップ⑥のX-Yクロスポイント画面に対応)

5. AliasConnector(Device配下、V/A/ANC最大3つの束)

項目名	型	説明
id	UUID (PK)	本システム内部管理用ID(NMOSには公開されない)
device_id	UUID (FK → AliasDevice)	所属Device
connector_label	string	ユーザー設定ラベル(例:「SB1」「KDDI1」)
created_at	datetime	作成日時

6. AliasSender(本システムの中核データ)

source_idを追加。

項目名	型	説明
-----	---	----

id	UUID (PK)	新規生成されるAlias Sender自身のNMOS Sender ID
connector_id	UUID (FK → AliasConnector)	所属Connector
real_sender_id	UUID (FK → RealSender)	紐付け元のReal Sender参照(多対1)
label	string	{Connector label}_{V/A/ANC}の形式で自動生成
description	string	他ゾーンRDSへ広告用の説明文(ユーザー変更可)
media_type	enum(video/audio/ancillary)	ユーザーが設定/上書き可能な分類。同一Connector内で重複不可
sdp_mirrored	text	real_senderのSDPをそのまま複製した内容(常時同期)
sync_status	enum(online/offline)	Real Senderとの同期状態(オフライン検知用)
last_synced_at	datetime	直近の同期日時
source_id	UUID	このAliasSenderに対応する、自動生成されるNMOS Source ID(1:1)。 label/descriptionはAliasSenderの値をそのまま流用(新規)

7. AliasSenderRegistration(AliasSenderの他ゾーンRDS登録状態)

項目名	型	説明
-----	---	----

id	UUID (PK)	
alias_sender_id	UUID (FK → AliasSender)	対象のAlias Sender
zone_rds_config_id	UUID (FK → ZoneRdsConfig)	登録先その他ゾーンRDS設定
registration_status	enum(online/offline)	このRDSへの登録状態(Real Senderオフライン時は自動でoffline)
last_registered_at	datetime	直近の登録(ハートビート)成功日時

8. SameZoneRdsConfig(同一ゾーンRDS接続設定)

項目名	型	説明
enabled	boolean	ON/OFF
ip_address	string	接続先IP
port	int	接続先ポート
query_api_version	string	IS-04 Query APIバージョン(1.1～1.3)

9. ZoneRdsConfig(他ゾーンRDS接続設定)

項目名	型	説明
id	UUID (PK)	

node_id	UUID (FK → AliasNode)	どのNodeに紐づく設定か
registration_api_enabled	boolean	Registration API ON/OFF
registration_ip_address	string	接続先IP
registration_port	int	接続先ポート
registration_api_version	string	IS-04 Registration APIバージョン

エンティティ関連図(改訂版)

SameZoneRdsConfig (1) — 監視対象 —> RealSender (N)

RealSender (1) — N — AliasSender

AliasNode (N) — N — AliasDevice ★NodeDeviceAssignment経由でM:N

AliasDevice (1) — N — AliasConnector

AliasConnector (1) — N(最大3) — AliasSender

AliasNode (1) — N — ZoneRdsConfig

AliasSender (N) — N — ZoneRdsConfig ★AliasSenderRegistration経由でM:N

AliasNode (1) — 1 — (node_api_enabled / node_api_port) ※Node自身が保持

AliasDeviceが複数のAliasNodeに紐づけ可能になったことで、そのDevice配下のConnector/Senderは複数の**Node**経由で複数の**RDS**に同時に登録されうることになります。旧モデルの **registered_to_zone_rds_id**(単一のnullable FK)ではこれを表現できないため、AliasSenderとZoneRdsConfigの間に中間テーブルを設け、**Sender**×登録先**RDS**ごとに個別の登録状態を持つようにしました。

- RealSenderのmedia_type_detectedは**SDP**から機械的に自動取得する値であり、改変しない(参照専用)
- AliasSender作成時、UIはRealSenderのmedia_type_detectedを初期値として表示する
- ただしAliasSender.media_typeはユーザーが独立して設定・上書き可能とし、real_senderの値追従は行わない(一度設定したらReal側が変わっても自動更新されない、という前提とする)

● ⑪外部インターフェース仕様(NMOS API)

1. 基本方針

本システムは「RDS機能を持たない」「Receiver機能を持たない」(④用語集)という前提のもと、5つのNMOS関連APIについて、本システムがクライアントとして叩く側か、サーバーとして提供する側かを明確に区別する。

さらに、1台の物理サーバー内に複数のAliasNodeが存在するため、**Node API**と**Connection API**は**AliasNode**ごとに割り当てた1つのポート番号で区別し、**/x-nmos/**配下の別パスとして同一**Web**サーバー上で共通提供する(⑩AliasNode.node_api_port に対応)。

2. API一覧と役割整理

No	API名	規格	本システムの役割	通信相手	対応バージョン	主な用途
1	Query API	IS-04	クライアント	同一ゾーンRDS	1.1 ~ 1.3	同一ゾーン内の本物Node/Device/Senderの一覧・詳細情報を取得
2	Query API WebSocket Subscription	IS-04	クライアント	同一ゾーンRDS	1.1 ~ 1.3	/subscriptions によるリアルタイム通知を受信し、AliasSenderのSDP同期トリガーとする
3	Registration API	IS-04	クライアント	他ゾーンRDS(AliasNode単位で複数設定)	1.1 ~ 1.3	AliasNode/AliasDevice/AliasSource/AliasSenderをリソース登録し、5秒間隔のハートビートで生存確認を継続
4	Node API	IS-04	サーバー(P2Pモード)	他ゾーンのNMOSコントローラー/RDSなしプロコン(LAWO VSM等)	1.1 ~ 1.3	他ゾーンRDSと別セグメントの相手からの静的な状態取得に対応
5	Connection API	IS-05	サーバー	他ゾーンのブロードキャストコントローラー/Receiver	1.0 ~ 1.1	AliasSenderのSDP(transportfile)取得、Receiver側のActivate操作への対応

整理の要点

- 本システムはRDS機能を持たないため、Registration APIとQuery APIは常にクライアント側
- Node APIとConnection APIは、他ゾーン側から見て「本物のNMOS機器のように振る舞う」必要があるため、本システムがサーバー側になる
- Node APIは、RDSを介さず直接SenderのSDP情報を取得するタイプのブロードキャストコントローラー(LAWO VSM等)にも対応するため、**/sources**エンドポイントもスコープに含める。本システムはReceiverを持たないため**/receivers**は対象外(空配列で応答)

3. AliasNodeスコープによるレスポンス制御

⑩でNode⇄DeviceがM:N(NodeDeviceAssignment経由)になったため、Node API・Registration APIともに**「どのAliasNodeから見るか」によって見えるDevice/Connector/Sender/Sourceの範囲が変わる**。

- 例:AliasDevice「キャリア」がAliasNode「報道用」とAliasNode「制作用」の両方に割り当てられている場合、両方のNode APIレスポンス(それぞれ別ポート)に同じAliasSender/AliasSourceが含まれる(NMOS ID自体は同一)
- Registration APIも同様に、AliasNodeごとに「そのNodeにひも付くDevice/Connector/Sender/Source」のみを対象ゾーンRDSへ登録する

4. 各APIの主要エンドポイント

① Query API(クライアント / 同一ゾーンRDS宛)

- GET /x-nmos/query/{version}/nodes
- GET /x-nmos/query/{version}/devices
- GET /x-nmos/query/{version}/senders
- GET /x-nmos/query/{version}/senders/{senderId}(SDP参照用の詳細取得)

② WebSocket Subscription(クライアント / 同一ゾーンRDS宛)

- POST /x-nmos/query/{version}/subscriptions(sendersリソースを対象に作成)
- 発行されたWebSocket URLに接続し、GRAINメッセージでSender変更を検知

③ Registration API(クライアント / AliasNode単位で他ゾーンRDS宛)

- POST /x-nmos/registration/{version}/resource(node/device/source/sender登録。AliasNodeごとにZoneRdsConfigで指定された送信先へ送信)
- POST /x-nmos/registration/{version}/health/nodes/{nodeId}(ハートビート、5秒間隔)
- 登録・登録解除の都度、AliasSenderRegistrationテーブルの該当行(alias_sender_id × zone_rds_config_id)を更新

④ Node API(サーバー / AliasNodeごとのポートでスコープされたレスポンス)

- GET /x-nmos/node/{version}/self
- GET /x-nmos/node/{version}/devices → そのAliasNodeにNodeDeviceAssignmentされたAliasDeviceのみ
- GET /x-nmos/node/{version}/sources → 該当Device配下のAliasConnector→AliasSenderに対応するAliasSourceのみ
- GET /x-nmos/node/{version}/sources/{id}
- GET /x-nmos/node/{version}/senders → 同様にスコープされたAliasSenderのみ
- GET /x-nmos/node/{version}/senders/{id}
- GET /x-nmos/node/{version}/senders/{id}/transportfile
- GET /x-nmos/node/{version}/receivers → 常に空配列(Receiver機能を持たないため)

⑤ Connection API(サーバー / Node APIと同一ポート・同一Webサーバー)

- GET /x-nmos/connection/{version}/single/sendernames/{id}/transportfile
- GET /x-nmos/connection/{version}/single/sendernames/{id}/staged
- PATCH /x-nmos/connection/{version}/single/sendernames/{id}/staged
- GET /x-nmos/connection/{version}/single/sendernames/{id}/active

5. ポート・接続方式のまとめ

項目	内容
Query API / WebSocket / Registration API	本システムがクライアント。接続先IP/ポート/バージョンはユーザーがGUIで設定(⑨)
Node API / Connection API	AliasNodeごとに1つの待受ポート(node_api_port)を割り当て、 /x-nmos/node/... と /x-nmos/connection/... のパスで両APIを共通提供
ハートビート間隔	IS-04標準に準拠し5秒間隔

6. Source/Senderの対応関係

- AliasSenderとAliasSourceは1:1(⑩AliasSender.source_id)
- Source側のlabel/descriptionはAliasSenderの値をそのまま流用し、独立編集項目は持たない
- NMOSの一般的なSource-Sender-Flow関係のうち、本システムはFlowリソースを独自に生成する必要はなく、Real SenderのSDPをそのままミラーすることでSource/Sender情報として提供する(SDPの改変を行わないという④⑤の大前提に準拠)

● ⑫機能要件一覧

A. 同一ゾーンRDS連携(Query API / WebSocket)

ID	要件	関連章
REQ-A01	起動時、SameZoneRdsConfigの設定に基づき同一ゾーンRDSのQuery APIへ接続し、Node/Device/Senderの一覧を取得してRealSenderテーブルに登録する	⑦①⑨
REQ-A02	取得したSenderごとにGET /senders/{senderId} でSDP詳細を取得し、RealSender.sdp_rawに保存する	⑪4-①
REQ-A03	SDPの内容からmedia_type(video/audio/ancillary)を自動判定し、RealSender.media_type_detectedに保存する(参照専用、改変しない)	④⑤⑩
REQ-A04	Query APIの /subscriptions を通じてsendersリソースをWebSocket購読し、 GRAIN メッセージでSender変更を検知する	④⑫-3、⑪4-②
REQ-A05	WebSocketでSender変更を検知した際、該当RealSenderのsdp_rawを即時更新し、紐づく全AliasSenderのsdp_mirroredに同期する	⑦③②、⑧ステップ⑤
REQ-A06	同一ゾーンRDS上でSenderが消失(登録解除)したことを検知した場合、該当RealSenderをstatus=offlineとし、紐づく全AliasSenderをsync_status=offlineにする	⑦③④

REQ-A07	消失していたSenderが復活した場合、RealSenderをstatus=onlineに戻し、紐づくAliasSenderのsync_statusとRDS登録状態を速やかに復活させる	⑦③④
---------	--	-----

B. AliasNode管理

ID	要件	関連章
REQ-B01	ユーザーがWebGUIから新規AliasNodeを作成できる(label / description / node_api_enabled / node_api_portを設定)	⑨「Alias Nodeの作成管理」
REQ-B02	1つのAliasNodeを複数のZoneRdsConfig(他ゾーンRDS)に登録できる(1:N)	④⑩-2
REQ-B03	1つのZoneRdsConfig(RDS)に対して複数のAliasNodeを紐づけることは不可とする(排他制御)	⑨「Alias Nodeの作成管理」
REQ-B04	AliasNodeの削除は、紐づいたZoneRdsConfigが存在しないことを条件とする	⑨「Alias Nodeの作成管理」

C. AliasDevice / AliasConnector管理

ID	要件	関連章
REQ-C01	ユーザーがWebGUIから新規AliasDeviceを作成・削除できる(label / descriptionを設定、慣習として「label:回線名」「description:回線種別」)	⑧ステップ③、⑨
REQ-C02	ユーザーがAliasDevice配下に複数のAliasConnectorを作成・削除できる(connector_labelを設定)	⑦②③、⑨
REQ-C03	X-Yクロスポイント画面で、AliasNode(X)とAliasDevice(Y)の紐づけ(NodeDeviceAssignment)を設定・解除できる。M:N関係で、1つのAliasDeviceを複数のAliasNodeに割り当て可能	④⑩-3、⑧ステップ⑥

D. AliasSender管理・SDP同期

ID	要件	関連章
REQ-D01	AliasSender作成時、同一ゾーン内RDSのNode/Device/Senderをプルダウンで選択し、対象のRealSenderを1つ選ぶ(文字列フィルター・media_typeフィルター対応)	⑨「Alias Sender設定」
REQ-D02	AliasSender作成時、紐づけ先のAliasConnectorを選択する。以後も紐づけ先Connectorの変更は可能とする	⑧ステップ④

REQ-D03	AliasConnectorに紐づいた時点で、AliasSender.labelを{Connector label}_{V/A/ANC}の形式で自動生成する。以後この画面でのlabel変更は不可	⑧ステップ④、⑨
REQ-D04	同一AliasConnector内で、media_type(video/audio/ancillary)が重複するAliasSenderの登録を禁止する(最大3つまで)	⑦②③、⑩
REQ-D05	AliasSender.media_typeの初期値はRealSenderのmedia_type_detectedを提案するが、ユーザーが独立して上書き設定できる(以後は本物側の変更に追従しない)	⑩
REQ-D06	AliasSender.descriptionは、Connectorとの紐づけ後も変更可能とする	⑨「Alias Sender設定」
REQ-D07	AliasSender作成時、対応するAliasSourceを自動生成する(source_id発行)。Source側のlabel/descriptionはAliasSenderの値をそのまま流用する	⑪6
REQ-D08	RealSenderのsdp_rawが更新された場合、紐づく全AliasSenderのsdp_mirroredをリアルタイムで同期する(SDP本体は改変しない)	⑦③②、④⑤

E. 他ゾーンRDS登録(Registration API)

ID	要件	関連章
REQ-E01	ユーザーがWebGUIからZoneRdsConfig(他ゾーンRDS接続先IP/ポート/APIバージョン)をAliasNode単位で複数登録できる	⑨「RDS登録管理」
REQ-E02	AliasNodeごとに、そのNodeにNodeDeviceAssignmentされたAliasDevice配下のAliasConnector/AliasSender/AliasSourceのみを対象ゾーンRDSへPOST /resourceで登録する	⑪3、⑪4-③
REQ-E03	登録済みのAliasNodeについて、5秒間隔でPOST /health/nodes/{nodeId}によるハートビートを送信し続ける	⑤⑤、⑪5
REQ-E04	AliasSenderのsync_statusがofflineになった場合、対応するAliasSenderRegistrationのregistration_statusを自動的にofflineへ変更し、他ゾーンRDSへの登録をOFFにする	⑦③④
REQ-E05	AliasSenderのsync_statusがonlineに復活した場合、対応するRDSへの登録(registration_status)を速やかに復活させる	⑦③④

REQ-E06	Registration APIへの登録・登録解除のたびに、AliasSenderRegistrationテーブル(alias_sender_id × zone_rds_config_id)の該当行を更新する	⑩⑦
---------	--	----

F. Node API(サーバー機能)

ID	要件	関連章
REQ-F01	AliasNodeごとに割り当てたnode_api_portでWebサーバーを立て、 <code>/x-nmos/node/{version}/self</code> を提供する	⑪4-④、⑩
REQ-F02	<code>/x-nmos/node/{version}/devices</code> は、リクエストを受けたAliasNodeにNodeDeviceAssignmentされたAliasDeviceのみを返す	⑪3
REQ-F03	<code>/x-nmos/node/{version}/sources</code> および <code>/sources/{id}</code> は、該当AliasNodeスコープ内のAliasSourceのみを返す	⑪4-④
REQ-F04	<code>/x-nmos/node/{version}/senders</code> および <code>/senders/{id}</code> は、該当AliasNodeスコープ内のAliasSenderのみを返す	⑪4-④
REQ-F05	<code>/x-nmos/node/{version}/senders/{id}/transportfile</code> で、対象AliasSenderのsdp_mirroredをtransportfile形式で返す	⑪4-④
REQ-F06	<code>/x-nmos/node/{version}/receivers</code> は常に空配列を返す	④⑨-2、 ⑪4-④

G. Connection API(サーバー機能)

ID	要件	関連章
REQ-G01	Node APIと同一ポート・同一Webサーバー上で <code>/x-nmos/connection/{version}/single/sendereports/{id}/transportfile</code> を提供する	⑪4-⑤、 ⑤
REQ-G02	<code>/staged</code> エンドポイントのGET/PATCHに対応し、Receiver側からのActivate操作(IS-05, IGMP Join Version3対象)を受け付ける	④③、 ⑪4-⑤
REQ-G03	<code>/active</code> エンドポイントで現在のアクティブな接続状態を返す	⑪4-⑤

H. WebGUI

ID	要件	関連章
REQ-H01	ダッシュボード画面で、同一ゾーン/他ゾーンRDSそれぞれの接続状態・IP・ポート・APIバージョンを表示する	⑨3-①

REQ-H 02	ダッシュボードで、同一ゾーンRDSから取得したNode一覧を表示し、クリックでDevice/Sender/Receiver状態を別ウィンドウ表示する	⑨3-①
REQ-H 03	Alias Node情報ペインで、Alias Device一覧をクリックで展開し、内部のAlias Connectorを表示、さらにConnectorをクリックすると内部の3つのAlias Senderの同期状態(ラベル・S,G状態)を1行ずつ表示する。SDPファイルの内容表示・ダウンロードにも対応し、全体をスクロール可能とする	⑨3-①(ペイン④)
REQ-H 04	RDS登録管理画面で、同一ゾーンRDS(Query API送信先)と他ゾーンRDS(Registration API送信先、複数登録可)のON/OFF・IP・ポート・バージョンを設定できる	⑨3-②
REQ-H 05	Alias Node作成管理画面で、Node label設定、他ゾーンRDSとの紐づけ、削除操作(紐づくRDSがないことが条件)を行える	⑨3-③
REQ-H 06	Alias Device/Connector管理画面で、Device/Connectorの作成・削除・label設定、およびX-Yクロスポイント画面でのNode-Device紐づけを行える	⑨3-④
REQ-H 07	Alias Sender設定画面で、対象RealSenderの選択・フィルタリング、Connectorとの紐づけ、description/media_typeの設定を行える	⑨3-⑤
REQ-H 08	システム設定画面で、データベースをエクスポート・インポートできる。PoCとしては、実体であるSQLiteのDBファイル(.db)そのものをダウンロード/アップロードする方式とする(形式変換の実装コストを避けるため)	⑨3-⑥
REQ-H 09	ダッシュボードおよびシステム設定画面で、本システムが稼働するサーバーのOS IPアドレスを取得・表示する(参照専用、本システムからの設定変更は行わない)	⑨3-①、 ⑨3-⑥

● ⑬非機能要件・スコープ外事項

非機能要件(PoCとして最低限考慮する事項)

ID	要件	理由・関連章
NFR-01	Dockerコンテナ再起動時、SQLiteファイルの内容(AliasNode/Device/Connector/Sender等の設定)は保持される(揮発しない)	⑤⑦⑧、再設定の手間を避けるため
NFR-02	コンテナ再起動後、既存のAliasNode/AliasSenderの設定に基づき、同一ゾーンRDSへの再接続とAlias再構築を自動的に行う	⑦①、運用継続性の最低限の担保

NFR-03	主要な処理(RDS接続失敗、WebSocket切断、SDP同期失敗、他ゾーンRDS登録失敗など)について、コンソール/ログファイルへの最低限のログ出力を行う(検証時のデバッグ用途)	PoCでの動作確認・トラブルシュートに必要
NFR-04	WebGUIは日本語表示を基本とする(多言語対応は行わない)	運用者が日本語話者である前提
NFR-05	WebGUIの動作確認は最新版のモダンブラウザ(Chrome等)1種類のみを対象とする(クロスブラウザ対応は行わない)	PoCの検証コストを抑えるため
NFR-06	同時に扱うReal Sender / Alias Senderの数は、検証環境の規模(数台～数十台程度)を想定し、大規模スケールでの性能は考慮しない	③「安定性は考慮せず」に対応
NFR-07	Dockerコンテナは <code>--network host</code> (ホストネットワークモード)で起動し、コンテナはホストのIPアドレスをそのまま使用してポートをbindする。これにより、Alias Nodeごとに動的に追加されるポート番号に対して、Dockerのポートマッピング設定変更を不要にする	⑤⑧、⑨「Node API設定」、⑩AliasNode.node_api_port。IPアドレスはOS側管理・本システムはポート番号のみ管理、という設計方針(⑤⑧)を実現するため

明示的なスコープ外事項(本番システムでは必要だが、PoCでは対象外)

ID	項目	備考
OOS-01	WebGUI・各NMOS APIへの認証・認可(ユーザー管理、アクセス制御)	③で明言された「セキュリティは考慮しない」に該当。HTTP平文・無認証でのアクセスを許容する
OOS-02	通信の暗号化(HTTPS/TLS化)	同上。すべてHTTP平文通信を前提とする
OOS-03	高可用性構成(冗長化、フェイルオーバー、複数台構成でのクラスタリング)	⑤②「本システム以外は存在しない」単体構成が前提
OOS-04	詳細な監査ログ・ログ基盤(ログの永続化・検索・保管ポリシー等)	NFR-03の最低限ログのみで代替する
OOS-05	負荷試験・パフォーマンスチューニング(大量Sender/高頻度更新時の性能保証)	③「安定性は考慮せず」に対応
OOS-06	バックアップの自動化・スケジューリング運用	REQ-H08の手動エクスポート機能のみで代替する

OOS-07	障害時の自動復旧・自己修復機構(異常検知後の自動リトライ以上の高度な回復処理)	③のスコープに対し過剰
OOS-08	他社NMOS実装(nmos-cpp以外)との相互接続性の網羅的な検証	③②「実際の放送機器との疎通確認」の範囲内で十分とし、幅広い相互接続保証はしない
OOS-09	IS-04/IS-05以外のNMOS仕様(IS-07イベント&チュアリー、IS-08チャンネルマッピング等)への対応	④で明記された対応範囲(IS-04, IS-05)outside
OOS-10	セキュリティ脆弱性診断・ペネトレーションテスト	③「セキュリティは考慮しない」に対応

● ⑭検証・受け入れ基準

基本方針

- 本PoCの成否は、「他ゾーンのReceiver/ブロードキャストコントローラーが、**Real Sender**に一切アクセスすることなく、**Alias Sender**経由でSDP情報を取得しIGMP Joinできるか」(③4)を最終ゴールとする
- 各基準は「①一連の正常系フロー」「②異常系(オフライン検知・復旧)」「③複数Node/LAWO VSM対応」の3段階で構成する

A. 同一ゾーンRDS連携

ID	検証項目	合格基準	関連REQ
AC-A01	同一ゾーンRDS(nmos-cpp等)への接続	起動後、SameZoneRdsConfigの設定に基づきQuery APIへ接続でき、既存のNode/Device/Senderの一覧がRealSenderテーブルに登録される	REQ-A01
AC-A02	SDP取得と自動判定	取得した各Real SenderについてSDPが正しく取得でき、media_type(video/audio/ancillary)が実際のSDP内容と一致した形で自動判定される	REQ-A02, A03
AC-A03	WebSocketによるリアルタイム同期	同一ゾーンRDS側でReal SenderのSDP内容(例: マルチキャストアドレス)を変更した際、数秒以内にRealSender.sdp_rawおよび紐づくAliasSender.sdp_mirroredが更新される	REQ-A04, A05
AC-A04	Real Sender消失時のオフライン化	同一ゾーンRDSからReal Senderが登録解除された際、対応するAliasSenderのsync_statusがofflineになり、他ゾーンRDSへの登録もOFFになる	REQ-A06, E04

AC-A 05	Real Sender復活時の自動復旧	消失していたReal Senderが同一ゾーンRDSに再登録された際、対応するAliasSenderが自動的にonlineへ復帰し、他ゾーンRDSへの登録も復活する	REQ-A 07, E05
------------	---------------------	--	------------------

B～C. AliasNode / AliasDevice / AliasConnector管理

ID	検証項目	合格基準	関連REQ
AC-B 01	AliasNodeの作成・複数RDS登録	WebGUIから作成したAliasNode 1つを、2つ以上の他ゾーンRDS(ZoneRdsConfig)に登録できる	REQ-B01, B02
AC-B 02	RDS:AliasNodeの排他制御	既に別のAliasNodeが紐づいているRDSに対し、別のAliasNodeを重複して紐づけようとした際、エラーとなり登録できない	REQ-B03
AC-B 03	AliasNode削除の制約	ZoneRdsConfigが紐づいたままのAliasNodeを削除しようとした際、削除が拒否される	REQ-B04
AC-C 01	X-Yクロスポイントによる多対多紐づけ	1つのAliasDeviceを2つ以上のAliasNodeに割り当てた際、両方のAliasNodeのNode APIレスポンスに同一のAliasSender/AliasSourceが含まれる	REQ-C03, F02～F04
AC-C 02	AliasConnectorのV/A/ANC制約	同一AliasConnector内でmedia_typeが重複するAliasSenderを登録しようとした際、エラーとなり登録できない(最大3つまで)	REQ-D04

D. AliasSender管理・SDP同期

ID	検証項目	合格基準	関連REQ
AC-D 01	Alias Senderのlabel自動生成	AliasConnectorに紐づけた際、labelが{Connector label}_{V/A/ANC}の形式で自動生成され、以後手動変更できない	REQ-D03
AC-D 02	SDPの非改変同期	AliasSender.sdp_mirroredの内容が、Real SenderのSDPと(media_type表現を除き)完全に一致している	REQ-D08 、④⑤
AC-D 03	Alias Source自動生成	AliasSender作成時、対応するsource_idが自動発行され、Node APIの/sourcesエンドポイントで参照できる	REQ-D07

AC-D04	media_typeの独立性	AliasSender.media_typeをユーザーが上書き設定後、Real Sender側のmedia_type_detectedが変化しても、AliasSender側は追従せず維持される	REQ-D05
--------	----------------	--	---------

E. 他ゾーンRDS登録

ID	検証項目	合格基準	関連REQ
AC-E01	Registration API登録	AliasNode配下のAliasDevice/AliasConnector/AliasSender/AliasSourceが、対象の他ゾーンRDS(nmos-cpp等)に正しくnode/device/source/senderとして登録され、RDS側のQuery APIで確認できる	REQ-E01, E02
AC-E02	ハートビート継続	登録後、5秒間隔でハートビートが送信され続け、他ゾーンRDS側でAliasNodeがオフライン扱いにならない	REQ-E03

F～G. Node API / Connection API(対他システム疎通)

ID	検証項目	合格基準	関連REQ
AC-F01	RDS経由でのSDP取得(標準NMOSコントローラー)	他ゾーンのNMOSコントローラー(RDS経由)から、Alias SenderのSDPをConnection API経由で取得できる	REQ-G01
AC-F02	RDSなしブロン(LAWO VSM等)対応	RDSを介さず、Node APIの/senders・/sources・/senders/{id}/transportfileに直接アクセスするタイプのブロードキャストコントローラーから、SDP情報を正しく取得できる	REQ-F01～F05
AC-F03	Receiver非対応の明示	/x-nmos/node/{version}/receiversにアクセスした際、常に空配列が返る	REQ-F06
AC-F04	最終ゴール: 実機でのIGMP Join成功	他ゾーンのReceiver(実機)が、Alias SenderのSDPを元にConnection APIの/staged→/activeを経てIGMP Join(Version3)を行い、Real Senderが送出する実際のST2110映像/音声/アンシラリストリームを実際に受信できる	REQ-G02, G03、③4(最終ゴール)

H. WebGUI

ID	検証項目	合格基準	関連REQ
AC-H01	ダッシュボード表示	同一ゾーン/他ゾーンRDSの接続状態、Node/Device/AliasSenderの一覧・同期状態が画面上で正しく確認できる	REQ-H01～H03
AC-H02	各種設定画面の一連の操作	RDS登録・AliasNode作成・AliasDevice/Connector作成・AliasSender設定を、WebGUI操作のみで一気通貫して行い、AC-F04の状態まで到達できる	REQ-H04～H07
AC-H03	DBエクスポート/インポート	システム設定画面からSQLiteファイルをダウンロードし、別環境(または初期化後の同一環境)にアップロードすることで、設定が復元される	REQ-H08

PoC全体の成功判定(最終基準)

以下がすべて満たされた場合、本PoCは「成功」と判定する:

1. **AC-F04(実機でのIGMP Join成功)** が達成されること — これが③に定義された本システムの最終ゴールそのものであるため、必須条件とする
2. AC-A01～A05(同一ゾーンRDS連携の正常系・異常系)がすべて満たされること
3. AC-C01(複数Node間でのDevice共有)またはAC-F02(LAWO VSM相当のRDSなしプロコン対応)の少なくとも一方が実機/実装で確認できること(両方揃えばなお良いが、PoCとしてはどちらか一方の実証で「有効性の実証」という目的は達成され则认为る)